Design Patterns Course

Learn, Build, Adapt

Session #10

2024

Irina Kamalova

VP Lead Software Engineer @ JPMC
ex-Revolut & ex-Yandex
Built systems to support 45M+ clients

Refreshing

- What is **Visitor Pattern** useful for?
- What is **Flightweight Pattern** useful for?
- What is **Interpreter Pattern** useful for?

10.1

Combining Patterns

Question

What patterns were used in example?

Could we solve the task with
Abstract Class in Java?

Question

How could we solve this task in **Python**?

10.2

Model View Controller

MVC

BULLET POINTS

- The Model View Controller (MVC) Pattern is a compound pattern consisting of the Observer, Strategy, and Composite Patterns.

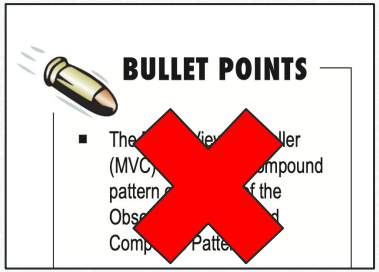
MVC

Irina: ***"They forgot to do a PEER REVIEW for this section!"***

BULLET POINTS

- The Model View Controller (MVC) Pattern is a compound pattern consisting of the Observer, Strategy, and Composite Patterns.

MVC



Reddit · r/learnprogramming

2 comments · 3 years ago

Why do all the backend developers choose controller- ...

Many people around me say the **controller-service-repository** pattern is **MVC**. As far as I know, the pattern **MVC**(model view **controller**) is just used for UI ...

Repository pattern is a pain. Are there better ways? - Reddit	5 Jan 2023
Is **Service** layer Part of **Controller** or Model in **MVC** - Reddit	25 Jul 2024
What is the difference between **Repository** and **Service** pattern?	25 Oct 2023
Examples of java projects using the **service**, model, **repository** ...	21 Apr 2021

More results from www.reddit.com

Source

MVC

Hacker News new | past | comments | ask | show | jobs | submit login

▲ danielovichdk on Feb 18, 2020 | parent | context | favorite | on: MVC is dead, it's time to MOVE on (2012)

Everyone who has done MVC long enough would also know that the controller is the very thin layer where services are injected and models are passes to.

The controller should have and only have logic which absolutely cannot be moved to a service layer.

Too many developers use controllers as part of their logic layer, instead of simply using it as a router for where models should be directed on a service level.

▲ jerf on Feb 18, 2020 | next [–]

"Everyone who has done MVC long enough... The controller should have and only have logic which absolutely cannot be moved to a service layer."

Sorry, which letter in MVC is the "service layer"?

One of the reasons I don't buy into MVC is that MVC doesn't mean *anything* any more, and hasn't ever since the web people got their hands on it. If you ask ten developers what MVC is and get 13 different answers (see half the sibling replies to this comment), well, that's not itself a problem, but it does mean that running around telling people that they should or should not do "MVC" has been pointless for a long time.

I still like MVC for its original purpose: Multiple views in a CAD/CAM program. Makes perfect sense there. I don't think I've yet seen a place where it belongs other than that, though.

▲ nck4222 on Feb 18, 2020 | parent | next [–]

MVC does not comprise all layers of an application. You might as well be asking which layer represents the repository layer while you're at it.

You can add other layers and frameworks in addition to using MVC. That doesn't mean MVC is pointless.

▲ jolmg on Feb 18, 2020 | root | parent | next [–]

> MVC does not comprise all layers of an application.

But the topic is MVC not some particular application. I think jerf's point is that it's weird to say that a service layer is an important part of working with MVC when it's not even part of the same idea. Your statement here confirms that.

One can be talking about an expansion of the idea of MVC as if it were MVC itself, but that just proves jerf's point, too, about MVC being meaningless.

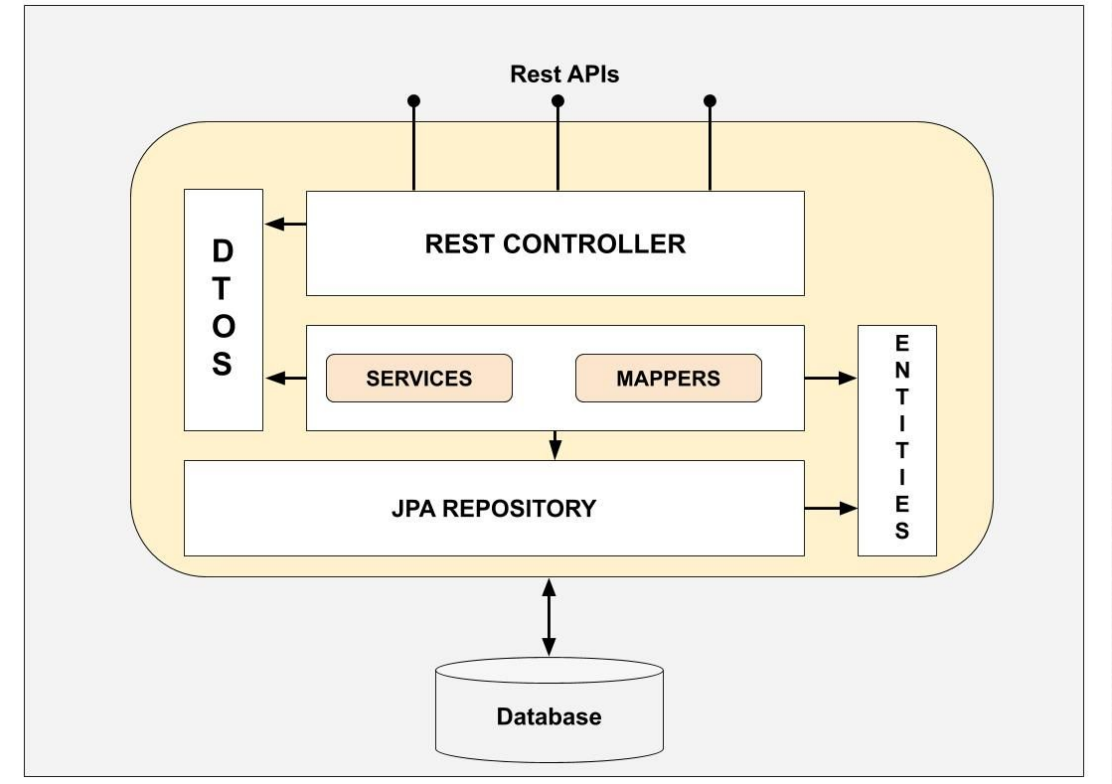
Source

Controller - Service - Repository

Controller	Management of the REST interface to the business logic
Service	Business Logic implementations
Repository	Storage of the entity beans in the system

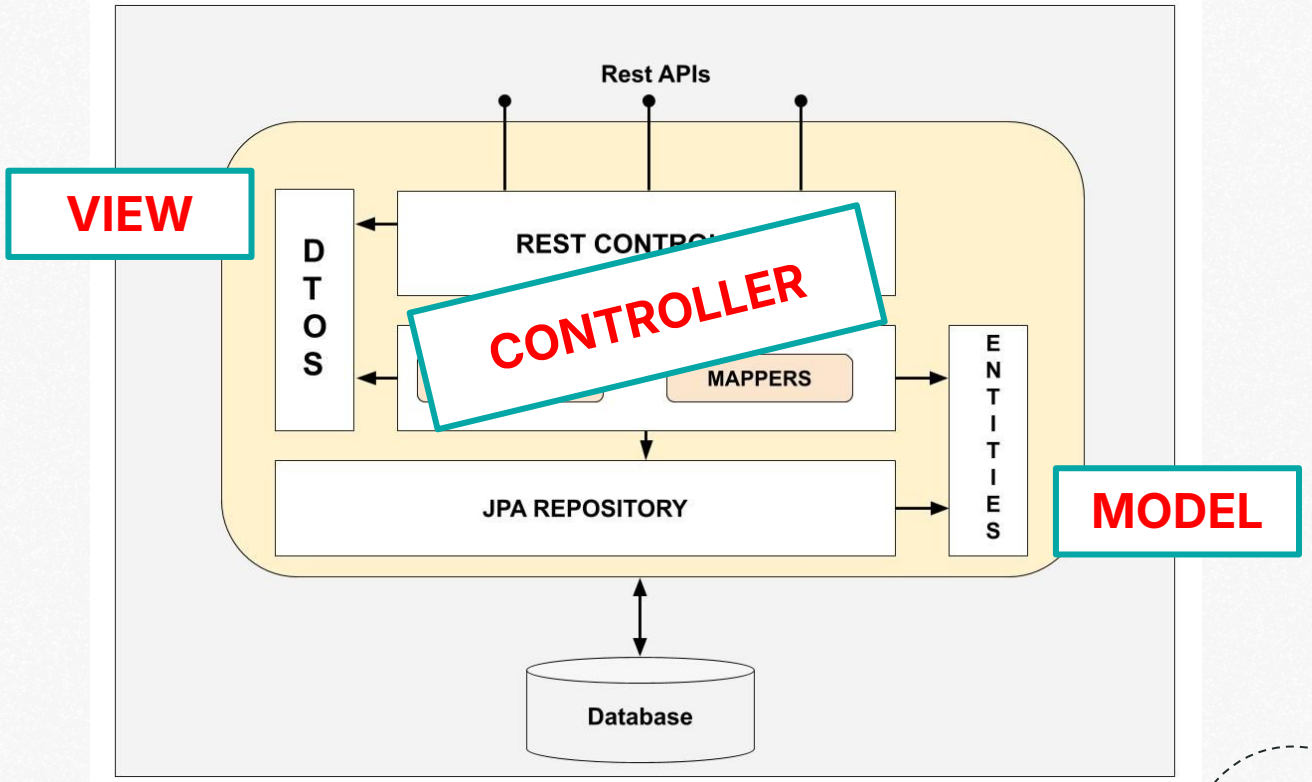
Source

Controller - Service - Repository



Source

Controller - Service - Repository



MVC

Model–View–Controller is a popular software pattern used to break up the logic of your application into three different components.

While the MVC pattern was initially used in desktop applications, it became popular to use in web applications during the late 1990's.

The Model is responsible for the data logic behind the application.

The View is what the user sees and interacts with in the application.

The Controller acts as the brains behind the application and communicates with the Model and View.

Web frameworks that use the MVC pattern include, Ruby on Rails, ASP.NET MVC, Laravel, and Angular.

No Observers

No Composites

SIMPLE!!!

Source

MVC

Model–View–Controller (MVC)

This doesn't appear in the original Design Patterns book.

There are a million different interpretations to MVC: from the original Xerox paper, from Ruby on Rails and other web frameworks, from JavaScript frameworks, and so forth. But in general:

Model: The source of truth for data; and the business logic, for example, the rules of a game.

View: The representation of the data as seen by the client. The client may be the user, or if we're talking about the server, the client is the browser, and the view is the mapping to JSON.

Controller: Handles the interactions from the client, and coordinates the work of the other parts of the system, including the Model and the View. Again, the client may be the user, or if we're talking about the server, the client is the browser which initiated the HTTP request.

Source

MVC vs Controller-Service-Repository

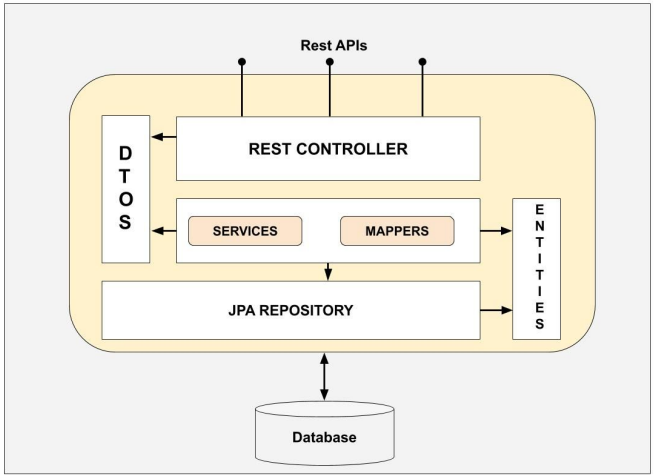
"It was a hard chapter to go through and now you're telling me "we don't use this MVC anymore" ???"

Ask all your colleagues and comrades devs how they understand MVC. Show them the book and bullet points. Align with their vision. Spike discussions. Find the harmony by being that much knowledgeable.

Controller - Service - Repository

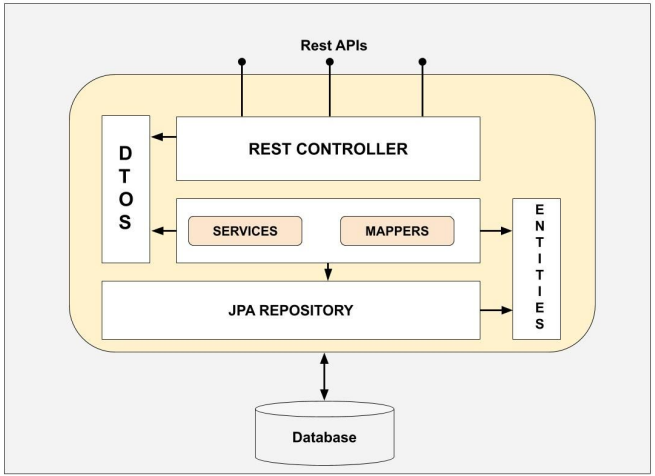
https://github.com/Women-Coding-Community/wcc-backend/blob/main/src/main/java/com/wcc/platform/controller/MemberController.java

Controller - Service - Repository



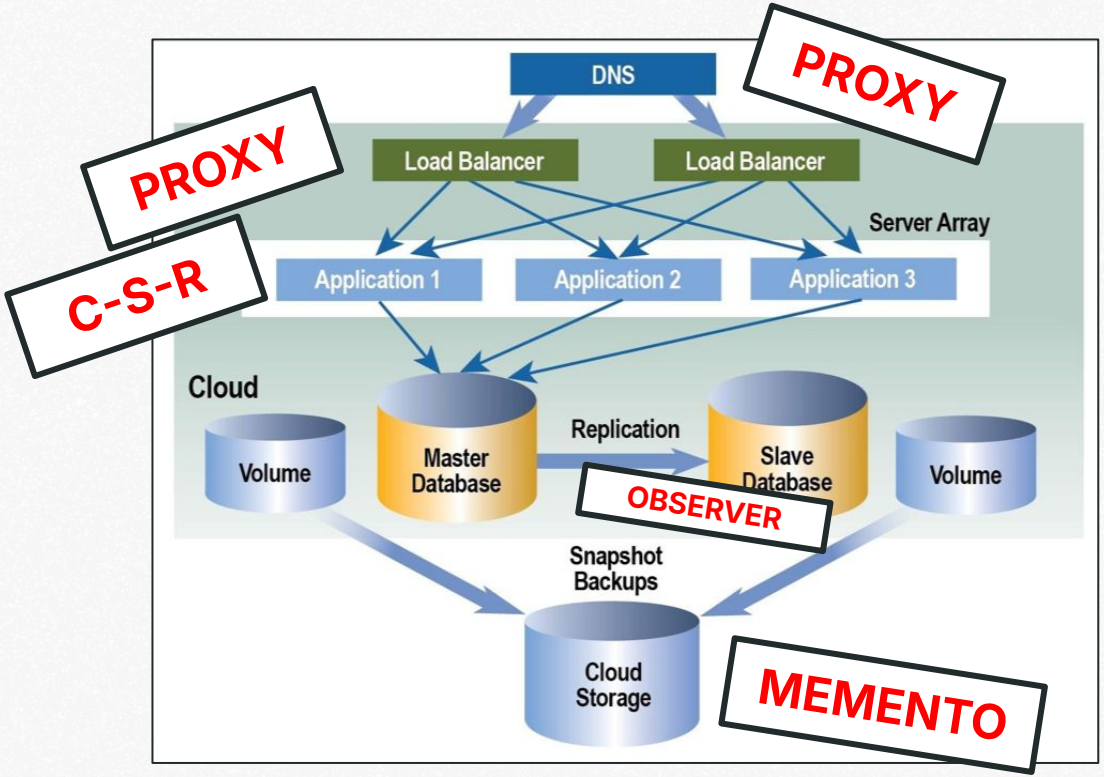
Abstract Factory & Factory Methods
- create controllers, services & repositories (& mappers)
C, S, R, M - singletons
Builders - create DTOs and Entities
Prototype - allow update Entities
State - keep your models in the proper state and control allowed operations

Controller - Service - Repository



Strategy - define different C, S, M, R
Decorators - encapsulate logging and metrics
Template Methods - encapsulate algorithms
Iterators or Streams - allow to work with collections
Proxy Caching - allow keep performance
Observers - allow to listen to events and update entities

Shifting to System Design



Source

Your Journey

01
Introduction
&
Strategy pattern

02
Observer
&
Decorator

03
Creational
Patterns

04
Command
&
Adapter

05
Facade
&
Template Method

06
Iterator
&
Composite

07
State
&
Proxy

08
Additional patterns:
Part I

Your Journey

09

Additional patterns: Part II

Combining patterns MVC

10

11

Surprise!

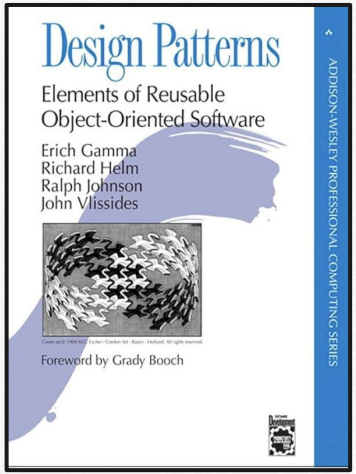
Resources

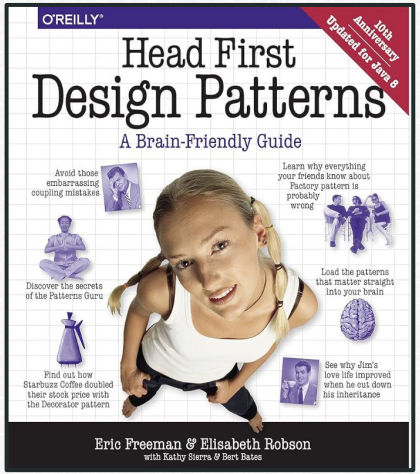
Books:

- Classic GOF Patterns
- Racoon Book
- Head First Design Patterns

Examples:

- Presentation (in Slack)





Thank you!